## Senior AI Data Software Developer Take-Home Test

### Context

Caseware builds software that supports audit, assurance, risk, and compliance professionals in delivering high-quality work efficiently and consistently.

As AI capabilities become increasingly important in audit workflows, we are investing in data platforms that allow AI agents to retrieve, reason over, and explain information from structured and unstructured business data.

This exercise evaluates your ability to design and implement a pragmatic local data pipeline and MCP server with good engineering judgment.

### The Problem

You are given a small procurement and inventory knowledge base containing:

### Document Categories

- Invoices
- Purchase Orders
- Shipping Orders
- Inventory Reports
- Contracts

The files include PDFs and image-based documents. Some documents may be easy to parse, while others may require extraction, OCR, or pragmatic handling decisions.

Your task is to build a local MCP server that allows ChatGPT or another MCP-compatible AI agent to ask questions over this knowledge base.

The system should help the AI agent retrieve relevant information, compare documents, reason across related records, and provide grounded answers with source references.

Example questions may include:

- “Which invoices are missing a matching purchase order?”
- “What purchase order supports this invoice?”
- “Does this shipment appear to match the related invoice?”
- “Summarize the contract terms relevant to supply of goods.”
- “Which documents support order 10687?”
- “Are there mismatches between invoices, purchase orders, and shipping orders?”
- “What inventory reports are available, and what period do they cover?”

- "Find evidence related to a specific vendor, order, or item."

### Expected Capabilities

Your solution should demonstrate, in a lightweight way:

- A local data pipeline for preparing the provided files
- Extraction and retrieval across PDFs and image-based documents
- Data modeling choices that support AI retrieval
- Embedding, indexing, OCR, text extraction, or search strategy
- MCP tools or resources that an AI agent can use effectively
- Grounded responses with source references or citations
- Basic document matching or cross-document reasoning where useful

You may decide how simple or advanced each capability should be within the time limit.

### Important Guidance

Please keep the solution simple and practical.

We are **not** looking for:

- Cloud deployment
- Production-grade infrastructure
- Complex distributed systems
- Excessive optimization
- A polished frontend

Prefer:

- Local execution
- Simple pipelines
- Clear architecture
- Maintainable code
- Reproducible setup

A clean, understandable, functional implementation is preferred over an overengineered one.

### MCP Server

Your MCP server should expose a small, well-designed set of tools and/or resources that allow an AI agent to query the knowledge base.

You should decide:

- What tools or resources to expose

- What parameters each tool accepts
- What structured results each tool returns
- How tools retrieve from invoices, purchase orders, shipping orders, inventory reports, and contracts
- How source references are returned
- How the server avoids returning unnecessary raw data

The MCP layer should consume processed or indexed data, rather than performing all extraction and transformation at request time.

### Data Pipeline and Modeling

You should design and implement a local pipeline that prepares the provided files for retrieval.

You should decide how to:

- Parse PDFs and image-based documents
- Extract text and relevant fields where useful
- Model relationships between invoices, purchase orders, shipping orders, inventory reports, and contracts
- Store embeddings, indexes, metadata, and source references
- Support retrieval across document categories
- Preserve enough lineage to trace answers back to original files
- Make the pipeline reproducible and re-runnable

We are interested in how you think about data architecture, not just whether the demo works.

### Retrieval Strategy

Your system should support retrieval across the provided document set.

You may choose your approach, including document search, vector search, keyword search, hybrid search, metadata filtering, graph-style relationships, field extraction, or tool-based routing.

You do not need to implement every strategy. Choose an approach that fits the problem and explain the tradeoffs.

### AI Usage

You must use Codex, Copilot, Claude Code, Cursor, ChatGPT, or other AI-assisted development tools.

Please be prepared to explain what parts were AI-assisted and how you validated the result.

### Time Expectation

Please spend no more than **3–5 hours** in total.

We intentionally want this exercise to remain lightweight. Do not overengineer the solution.

### Deliverables

Please submit:

- Source code repository
- README with setup instructions
- Instructions for running the data pipeline
- Instructions for running and connecting to the MCP server
- Brief architecture overview
- Brief data flow overview
- Explanation of data modeling choices
- Explanation of retrieval strategy
- Description of MCP tools or resources
- Citation or source-reference strategy
- Evidence that the MCP server works with an MCP-compatible client

### Presentation Requirement

Please prepare a short presentation of **10–15 minutes** covering:

- Software architecture
- Data pipeline architecture
- MCP server architecture
- AI agent / MCP interaction flow
- Tech stack decisions
- Data modeling
- Ingestion and retrieval strategy
- Structured extraction vs document retrieval approach
- Cross-document matching strategy
- Citation or source-reference strategy
- Future improvements

We want to understand not only what you built, but how you think about building AI-ready data platforms that support agentic workflows, retrieval systems, and knowledge management at scale.